

RFC BANCO - Bootcamp QI Tech

TIME	Matheus Silveira Feitosa, Rafael dos Santos Lombardi e Ryan de Oliveira Freitas
GRUPO	34
DATA	12/10/2026
VERSÃO	32

Contextualização

Entendendo o problema

O desafio era o de criar um banco digital que suportasse as operações de cadastro de clientes, criação de contas, transações, tarifas e extrato, diante disso, os principais usuários esperados deste sistema são, em especial, clientes PF e PJ que poderão:

- Cadastrar **clientes**
- Abrir **contas**
- Movimentar dinheiro entre contas **(transação,** mais amplo que transferência)
  - Que podem ser dentro do próprio banco ou envolver um banco externo.
    - A tarifa sobre a transação será cobrada de forma dinâmica a depender dos tipos de cliente envolvidos, canal de transação, e valor recebido.
- Obter um extrato de suas compras paginado pegando as 20 transações mais recentemente registradas no banco. (default 20, $1 \leq i \leq 100$)

O serviço precisa fornecer as seguintes garantias:

- Garantir que as transações ocorram de forma correta e o dinheiro seja rastreável.
- Nenhuma conta pode ficar negativa.
  - O que implica que devemos garantir que uma transferência só possa ser feita quando houverem fundos para tal. O que inclui casos de duas ou mais transferências simultâneas serem feitas simultâneamente sob o mesmo caso base.
- Um duplo clique ou uma retentativa não paga duas vezes.
- Transferências de dinheiro entre bancos só podem ser consolidadas quando o outro banco confirmar o sucesso da operação no lado deles.
- Nenhuma operação consolidada pode ser apagada do banco, mesmo em casos de inativações ou bloqueios.

Fora do escopo:

- Operações de crédito
- Estorno de transações consolidadas
- Operações de marcação de tempo
- Recebimento de instituições externas

Explicando a solução de forma macro

A API persiste os clientes em `client`, as contas e seus saldos atuais em `account`, as ordens financeiras em `bank_transaction` e as movimentações assinadas em `account_entry`.

O saldo da conta é materializado em **centavos** para permitir consultas rápidas. Os lançamentos funcionam como o histórico financeiro e apontam para a transação que os originou.

Nas transferências internas, a API trava as duas contas em ordem crescente de
id, refaz as validações com as contas travadas e grava a transação, os saldos

e os lançamentos no mesmo commit.

Nas operações externas, a validação inicial ocorre sem lock. Depois, a API trava a conta remetente, refaz as validações, considera as transações de saída `EM_ANDAMENTO` como reservas lógicas e grava a transação pendente. Esse estado é confirmado antes da chamada ao banco externo, liberando a conta durante a comunicação.

Se o banco externo responder `SETTLED`, a API trava a transação e a conta remetente, debita o valor mais a tarifa, cria o lançamento e marca a operação como `CONCLUIDA` no mesmo commit.

Se o banco externo rejeitar, a transação é marcada como `FALHOU` sem criar movimentação local. Se ocorrer timeout, a transação permanece `EM_ANDAMENTO`. O cliente deve repetir a mesma chave de idempotência, e a API consulta o identificador externo para reconciliar o resultado sem criar uma segunda operação.

A chave de idempotência é um UUID recebido no header `chave_de_idempotencia`. Ela é persistida diretamente, sem hash, e possui restrição única no banco. O identificador enviado ao banco externo é a `chave_transacao` local, armazenada como `external_transaction_id`.

A tarifa é calculada pelo canal, pelo tipo de cliente do remetente e pelo valor da operação. Não há diferenciação por horário.

Alternativas descartadas:

- **Filtro de extrato por período:** O contrato atual usa paginação por cursor, com `created_at` e `id`, em vez de paginação por número de página ou filtro temporal. Essa escolha mantém a posição do cliente estável quando novos lançamentos entram no topo do extrato. Um filtro por período pode ser adicionado futuramente com os índices adequados; ele não foi incluído por não fazer parte do escopo atual.
- **Recebimento de outro banco:** O fluxo `banco externo → conta interna` foi retirado do escopo. Implementá-lo exigiria uma entrada específica, referência externa, confirmação da liquidação, idempotência para créditos recebidos e um fluxo de devolução. A API atual implementa apenas transferências internas e envios para outro banco.
- **Saldo calculado pela soma do histórico:** O saldo não é recalculado somando todos os lançamentos a cada consulta. A conta mantém o saldo atual materializado, enquanto `account_entry` preserva o histórico. Isso permite consultas constantes do saldo e mantém a auditoria separada do estado atual. O saldo e os lançamentos são atualizados atomicamente na mesma transação SQL.

Implementação

Rotas

MÉTODO	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS QUANDO)
`GET`	`/`	Diz qual serviço é este. Aberta	nenhuma	`200`
`GET`	`/health_check`	Diz se o serviço está no ar. Aberta	nenhuma	`204`
`POST`	`/cliente`	Cadastra cliente único no banco de dados.	`documento` (11 dígitos no CPF, 14 no CNPJ), `nome`, `razao_social` (somente PJ), `nascimento` (somente PF)	`201` com `chave_cliente`; `200` com a `chave_cliente` existente, mesmo dados; `400` `QIT000001` corpo fora do formato; `409` `QIT001004` documento

MÉTODO	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS QUANDO)
				cadastrado com dados inválidos; `422` `QIT001003` dígito verificador errado; `422` `QIT001006` PF menor de 18 anos
`GET`	`/cliente/{chave_cliente}`	Devolve o cliente e as contas dele	`chave_cliente` no caminho	`200` ; `404` `QIT001001` client inexistente
`POST`	`/cliente/{chave_cliente}/conta`	Abre conta com estado `PENDENTE` .	`agencia` , `numero` (definidos pelo backoffice da agência; o `banco_id` é o do próprio banco)	`201` com `chave_conta` ; `20` com a `chave_conta` existente, mesma agência e conta deste cliente; `40` `QIT000001` ; `404` `QIT001001` ; `409` `QIT001011` agência e conta c outro cliente; `409` `QIT001012` client com 5 contas abertas
`GET`	`/conta/{chave_conta}`	Devolve a conta, com `EstadoConta` e `saldo`	`chave_conta` no caminho	`200` ; `404` `QIT001001` conta inexistente
`PATCH`	`/conta/{chave_conta}`	Muda o estado de `PENDENTE` para `ATIVA` , entre `ATIVA` e `BLOQUEADA` , e de qualquer um deles para `DESATIVADA` , que é final.	`novo_estado` ; `razão` (usada para fins de registro de eventos)	`200` ; `400` `QIT000001` ; `404` `QIT001001` ; `409` `QIT001008` transição fora da lista; `409` `QIT001009` desativar com saldo; `409` `QIT001010` desativar com transacao `EM_ANDAMENTO`
`POST`	`/transacao`	Move dinheiro entre duas contas internas ao banco, de uma conta para outra de outro banco ou de outro banco para uma conta do nosso banco, cobrando a tarifa do remetente além do valor transferido.	cabeçalho `chave_de_idempotencia` (UUID), `remetente` (informado através da `chave_conta`) se interno ao banco, `{banco_id, agencia, conta}` se de fora), `destinatario` (`{banco_id, agencia, conta}`), `valor` (centavos), `canal` (`PIX` , `TED` ou `BOLETO`).	`201` com `chave_transacao` `estado` , `valor` , `tarifa_cobrada` ; `202` no envio ser resposta, mesmo corpo com `EM_ANDAMENTO`
`GET`	`/conta/{chave_conta}/transacao/{chave_transacao}`	Devolve detalhes da transação.	as duas chaves no caminho	`200` ; `404` `QIT001001` conta inexistente; `404` `QIT001029`

MÉTODO	CAMINHO	O QUE FAZ	ENTRADA (CAMPOS QUE IMPORTAM)	SAÍDAS (STATUS QUANDO)
				transação inexistente
`GET`	`/conta/{chave_conta}/transacao`	Extrato, contendo a transação mais nova até a mais antiga de forma paginada.	`limite` (mínimo 1, padrão 20, maximo 100), `próxima`	`200` com `transacoes` e `proxima`, `null` r última página; `40` `QIT000001`; `400` `QIT001030` curso inexistente ou de outra conta; `404` `QIT001001`
`POST`	`/boleto`	Emite um boleto para uma conta beneficiária interna.	beneficiário, valor, data_vencimento, pagador; header contém chave_de_idempotencia	201 boleto registrado; 200 repetição idempotente; 40(QIT000001; 404 QIT001005; 409 QIT001017 ou QIT001022; 422 QIT001025; 504 QIT001028.
`POST`	`/boleto/{chave_boleto}/pagamento`	Paga e liquida um boleto externo ou emitido internamente.	Caminho: chave_boleto, corpo: pagador; header contém chave_de_idempotencia	201 pagamento concluído; 200 retry/reconciliaçã 404 QIT001024; 409 QIT001025, QIT001026, QIT001027 ou QIT001030; 422 QIT001021 ou QIT001029; 504 QIT001028.

Erros das rotas (Validações e regras de negócio)

Erros gerais e de validação

Status	Código	Quando acontece	O que o cliente faz
400	`QIT000001`	Corpo, header ou parâmetro fora do formato esperado	Corrige a requisição e envia novamente
403	`QIT000002`	`INTERNAL-TOKEN` ausente ou inválido	Envia o token correto

Clientes e contas

Status	Código	Quando acontece	O que o cliente faz
404	`QIT001001`	Cliente inexistente	Corrige a chave do cliente
404	`QIT001005`	Conta inexistente	Corrige a chave da conta
409	`QIT001006`	Abertura de conta com agência e número pertencentes a outro cliente	Escolhe outra agência e número
409	`QIT001007`	Cliente já possui cinco contas não desativadas	Não abre outra conta até desativar uma existente
409	`QIT001009`	Conta com saldo ao tentar desativá-la	Zera o saldo antes de desativar
409	`QIT001010`	Conta com transação em andamento ao tentar desativá-la	Aguarda a conclusão ou falha da transação

> Os códigos `QIT001011` e `QIT001012` não são usados pela implementação atual. Os códigos correspondentes são `QIT001006` e `QIT001007`.

Transferências PIX/TED

Status	Código	Quando acontece	O que o cliente faz
404	`QIT001013`	Conta remetente inexistente	Corrige a conta remetente
404	`QIT001014`	Conta destinatária interna inexistente	Corrige os dados da conta destino
404	`QIT001015`	Conta destinatária inexistente no banco externo	Corrige os dados enviados ao outro banco
409	`QIT001016`	Conta remetente não está `ATIVA`	Não repete até a conta estar ativa
409	`QIT001017`	Conta destinatária interna está `PENDENTE` ou `DESATIVADA`	Não repete até a conta poder receber
409	`QIT001018`	Conta destinatária externa não está ativa	Não repete enquanto o outro banco não disponibilizar a conta
422	`QIT001021`	Saldo menor que valor, tarifa e reservas de saídas em andamento	Não repete até haver saldo suficiente
422	`QIT001022`	Chave de idempotência usada com outra requisição	Não reutiliza a chave; usa outra apenas para uma nova operação intencional
422	`QIT001023`	Remetente e destinatário são a mesma conta	Corrige a conta destinatária
422	`QIT001019`	Banco externo rejeitou o envio	Não repete automaticamente; a ordem foi rejeitada
504	`QIT001020`	Banco externo não respondeu dentro do timeout	Repete o envio com a mesma chave de idempotência

Idempotência no timeout

No timeout, a API já pode ter gravado a transação local como `EM_ANDAMENTO`. O timeout não significa que o banco externo rejeitou a operação.

O cliente deve guardar a chave de idempotência e reutilizá-la no retry. A API consulta a operação externa pelo mesmo identificador e conclui a transação se ela estiver `SETTLED`, sem criar uma nova movimentação.

Não existe retry automático em background.

Boletos

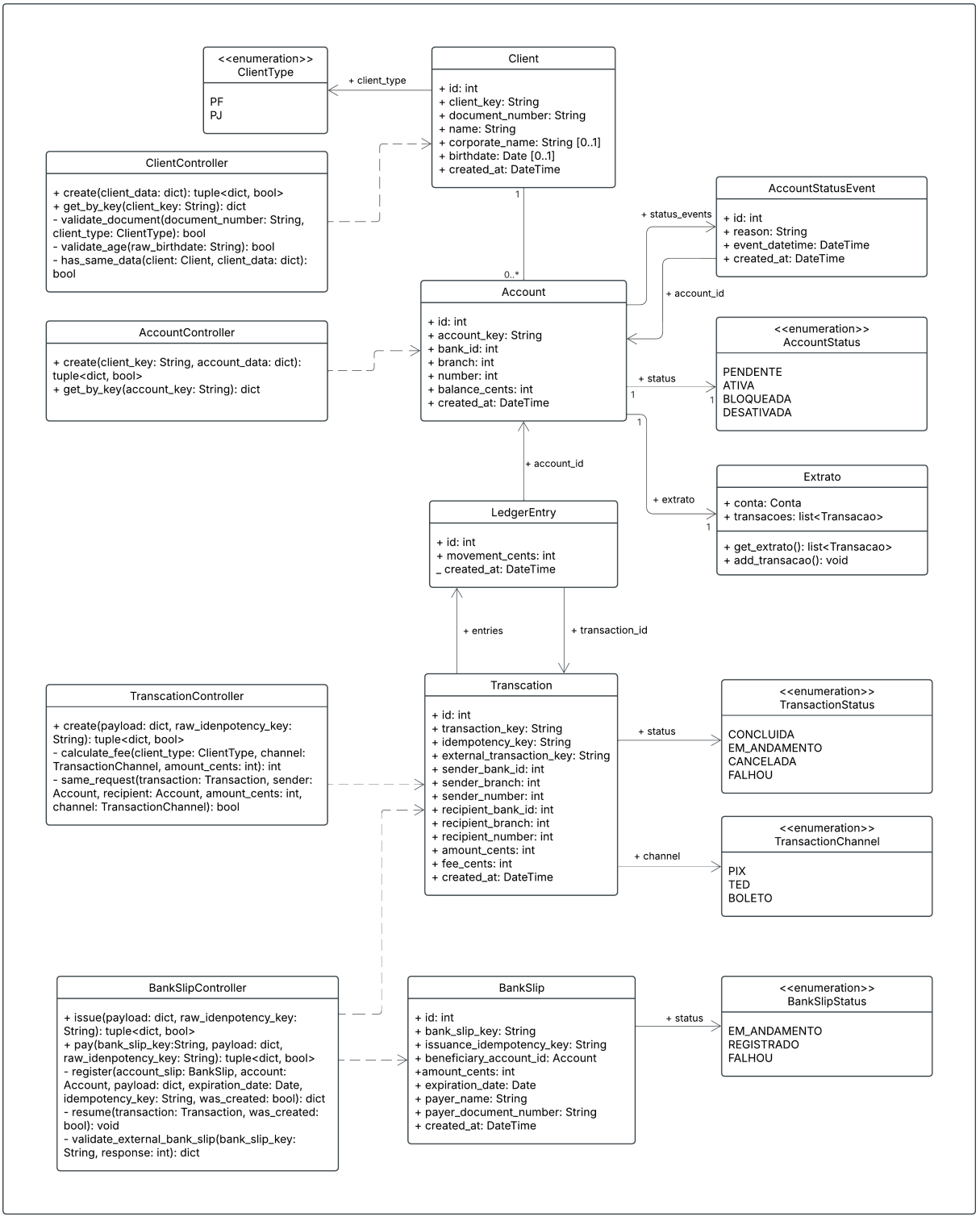
Status	Código	Quando acontece	O que o cliente faz
404	`QIT001024`	Boleto inexistente	Corrige a chave do boleto
409	`QIT001025`	Boleto vencido	Não tenta pagar o boleto vencido
409	`QIT001026`	Boleto já pago	Não repete o pagamento
409	`QIT001027`	Boleto possui pagamento incompatível em andamento	Não cria outro pagamento para o mesmo boleto
504	`QIT001028`	Serviço de boletos não respondeu	Repete o pagamento com a mesma chave de idempotência
422	`QIT001029`	Serviço de boletos rejeitou a operação	Não repete automaticamente
409	`QIT001031`	Beneficiário externo diverge do beneficiário registrado localmente	Interrompe a operação e corrige a associação

`QIT001026` e `QIT001027` não devem ser usados para representar uma entrada de outro banco. Esse fluxo não existe na API atual.

Extrato

Status	Código	Quando acontece	O que o cliente faz
400	`QIT001030`	Cursor `proxima` inexistente ou pertencente a outra conta	Reinicia a paginação ou usa o cursor correto

Diagrama de Classes (Banco de dados)



Fluxos

Cadastro de clientes

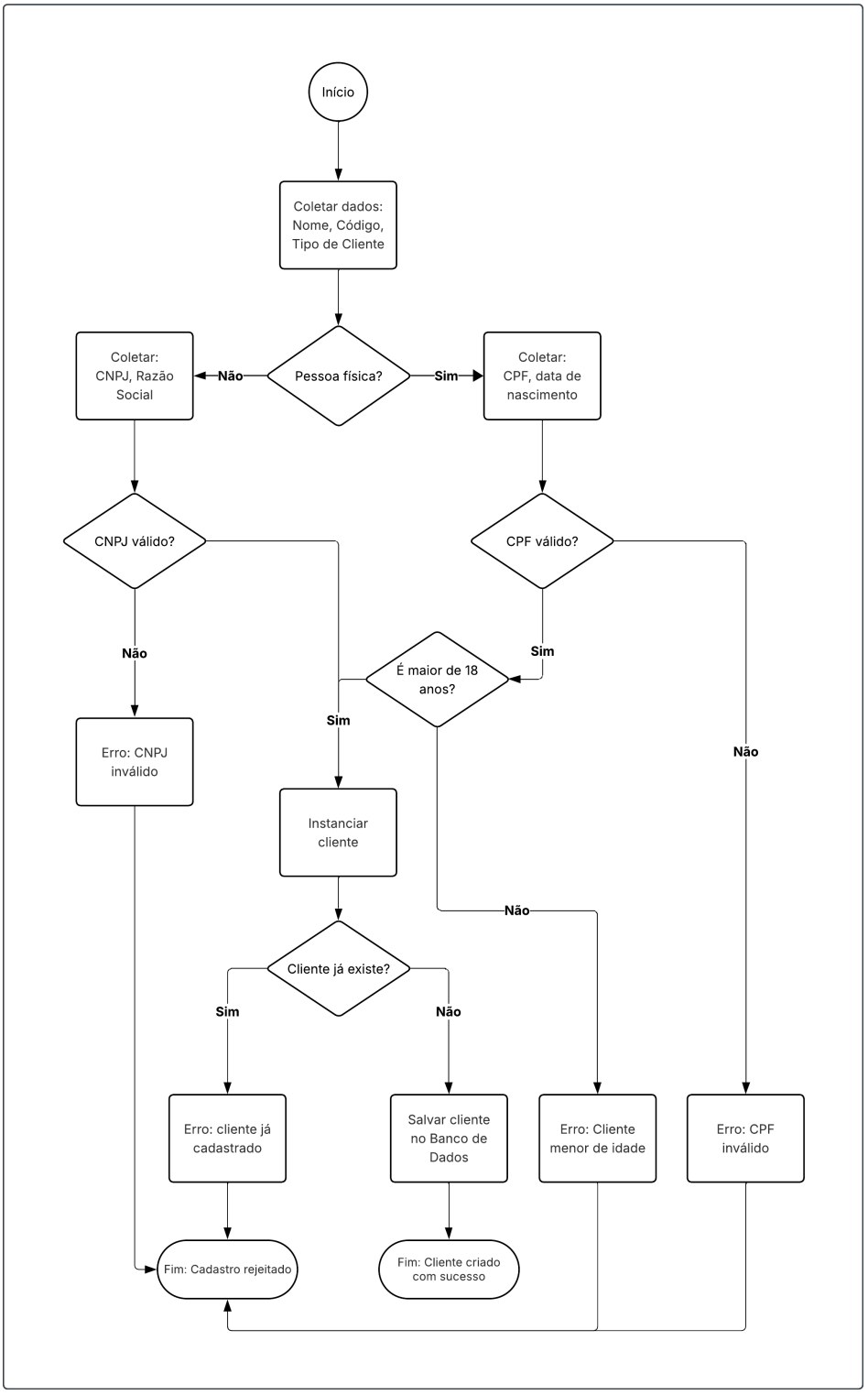
Cadastrar cliente, caminho feliz

1. Confere os dígitos verificadores do CPF, ou do CNPJ.
2. Exige 18 anos do cliente PF.
3. Grava a nova conta única no banco.

Cadastrar cliente, falha: o mesmo documento ao mesmo tempo

1. O `INSERT` do segundo pedido espera no índice único até o primeiro confirmar e não devolve linha.
2. A API lê o cliente gravado e responde com `409` `QIT001004` se não são.

Diagrama de fluxo do cadastro de clientes



Abertura de contas

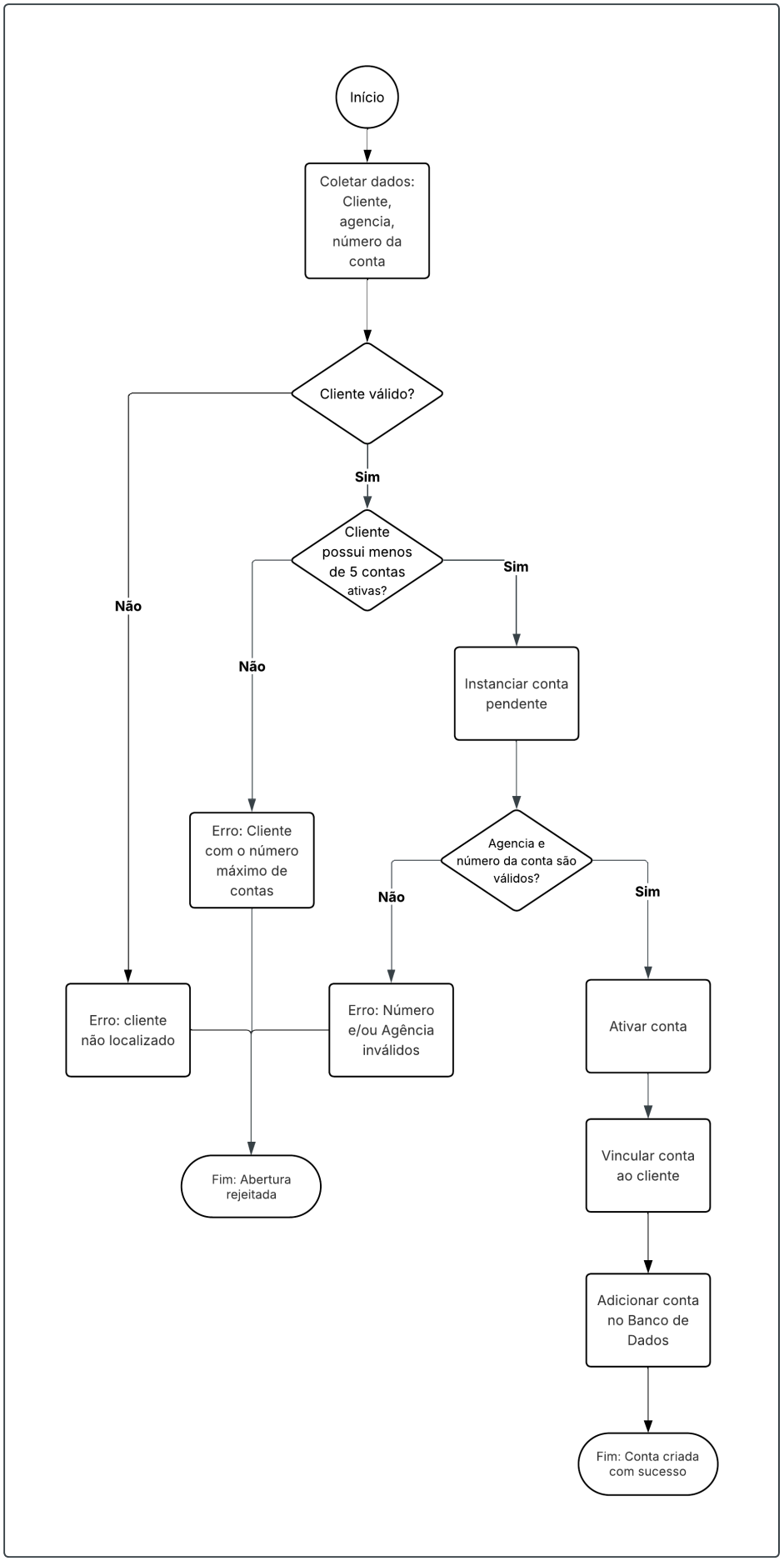
Abrir conta, caminho feliz

1. Trava a linha do cliente e procura a agência e conta informadas, que vêm do backoffice da agência. Agência e conta deste cliente devolvem `200` com a `chave_conta` existente, e agência e conta de outro cliente, `409` `QIT001011`.
2. Verifica quantas contas do cliente não estão em `DESATIVADA`. Escolhemos um teto de 5, no parâmetro `MAX_CONTAS_ABERTAS`, contra contas de passagem num mesmo documento. Com 5, responde `409` `QIT001012`.
3. Grava a conta em `PENDENTE` com `ON CONFLICT (agencia, numero) DO NOTHING` e o primeiro `evento_estado_conta`, e responde `201` com a `chave_conta`. Sem linha devolvida, outro cliente pegou a mesma agência e conta no mesmo instante, e a resposta é `409` `QIT001011`.

Abrir conta, falha: dois pedidos pela quinta conta

1. O cliente t|em 4 contas abertas, e os pedidos A e B chegam juntos.
2. A trava o cliente, grava a quinta conta e confirma, enquanto B espera.
3. B obtém a trava, encontra 5 contas abertas e responde `409` `QIT001012`.

Diagrama de fluxo da abertura de conta


Início
Coletar dados: Cliente, agencia, número da conta
Cliente válido?
Não
Sim
Cliente possui menos de 5 contas ativas?
Não
Sim
Erro: cliente não localizado
Erro: Cliente com o número máximo de contas
Instanciar conta pendente
Agencia e número da conta são válidos?
Não
Sim
Erro: Número e/ou Agência inválidos
Ativar conta
Fim: Abertura rejeitada
Vincular conta ao cliente
Adicionar conta no Banco de Dados
Fim: Conta criada com sucesso

Transações (PIX/TED)

Decisão de projeto

- Para toda nova operação de saída, o sistema trava a conta remetente, considera todas as saídas EM_ANDAMENTO como reservas e valida:

  saldo contábil >= reservas existentes + novo valor + tarifa

- O fluxo de recebimento de outro banco (externa → interna) está fora do escopo desta versão.
  - O POST /transacao sempre exige uma conta remetente pertencente ao nosso banco.
  - Essa decisão evita implementar agora endpoint de entrada, referência externa, confirmação do banco remetente, crédito local e fluxo de devolução. O external-bank-fake pode manter recursos auxiliares para testes, mas eles não representam uma rota pública da nossa API.

Transferir, cálculo da tarifa

A tarifa é calculada pelo controller com as contas já travadas, antes da conferência de saldo, a partir do `canal`, da condição do remetente e do `valor`. O horário da operação não muda a tarifa.

A tarifa é calculada antes da movimentação dos saldos, sem modificá-los diretamente. O valor resultante é registrado em `tarifa_cobrada`.

Os valores abaixo são decisões fictícias do banco, inspiradas nas práticas bancárias brasileiras.

Canal	Condição	Tarifa	Responsável
PIX	Remetente PF	Isento	—
PIX	Remetente PJ	0,5%, limitado a R$ 10	Remetente
TED	PF ou PJ	R$ 10 fixos	Remetente

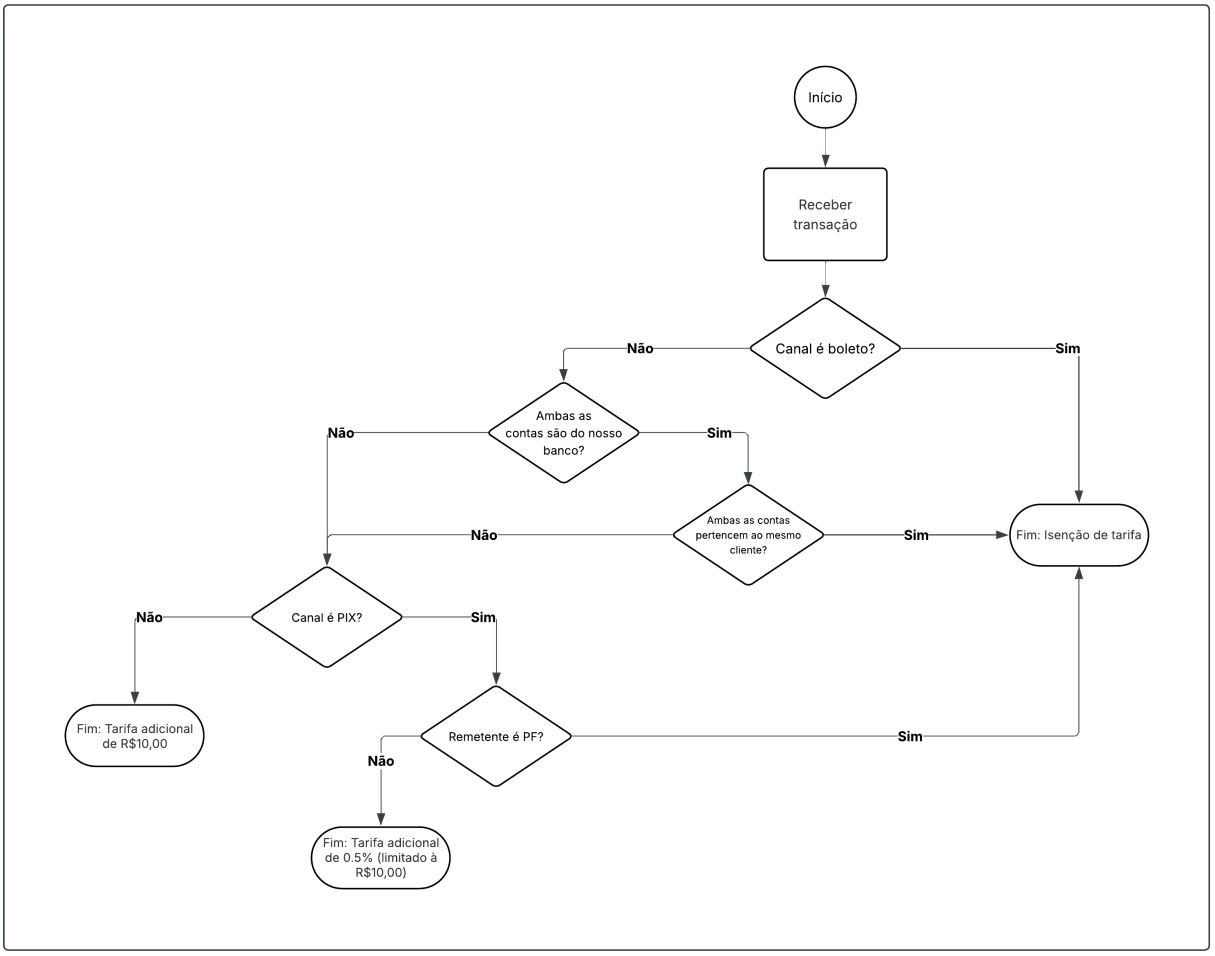
Isenções e condições adicionais:

- Se a transação for feita entre duas contas do nosso banco que pertencem ao mesmo cliente, não é cobrada tarifa.
- Tarifas não explicitamente previstas são consideradas isentas.
- Não haverá diferenciação tarifária por horário. Eventuais restrições operacionais serão tratadas separadamente.

O cálculo segue o diagrama abaixo:

1. Entrada vinda de outro banco não paga tarifa, porque o remetente não é cliente daqui.
2. Envio e transferência interna partem do `canal`: `TED` custa R$ 10,00 fixos.
3. Em `PIX`, a tarifa depende do `tipo_cliente` do remetente:
   - `FISICO`: isenta.
   - `JURIDICO`: 0,5% do `valor`, limitada a R$ 10,00.
4. O remetente paga o `valor` acrescido da `tarifa_cobrada`, e o destinatário recebe o `valor` integral. Sem saldo para os dois, a resposta é `422` `QIT001021`.
5. A tarifa é gravada em `tarifa_cobrada` e no lançamento `TARIFA`, no mesmo commit do `DEBITO` e da transação.

Diagrama de fluxo do cálculo da tarifa



Operações de transação

O fluxo de transação vale para dois casos:

- transferência entre duas contas internas;
- envio de PIX/TED de uma conta interna para outro banco.

O recebimento de outro banco (`externa → interna`) está fora do escopo desta versão. O `POST /transacao` sempre recebe uma conta remetente pertencente ao
nosso banco.

As validações seguem esta ordem. Falhas de validação não gravam nada. A exceção é o envio externo: a API grava `EM_ANDAMENTO` antes de chamar o banco
externo, pois a resposta pode sofrer timeout.

1. A conta remetente local precisa existir e estar `ATIVA`.
2. O destinatário interno precisa existir e estar `ATIVA` ou `BLOQUEADA`.
3. O destinatário externo precisa ser validado pelo banco externo.
4. O remetente local precisa ter saldo para `valor + tarifa`, considerando também as transações de saída `EM_ANDAMENTO`.
5. Com tudo válido, a API cria a transação e efetua a movimentação local.

Transferir entre duas contas internas, caminho feliz

1. A API procura a `chave_de_idempotencia` existente e valida o payload.
2. Trava as duas contas em ordem crescente de id.
3. Com as contas travadas, refaz as validações de estado e saldo.
4. Cria a transação `CONCLUIDA`, atualiza os saldos e cria os lançamentos assinados em `account_entry`.
5. A transação e os lançamentos são confirmados no mesmo commit.

6. Responde `201`.

A operação é protegida pelo lock das contas e pela restrição única da chave de idempotência.

Transferir, falha: duas transferências contra o mesmo saldo

A trava da conta impede que duas operações leiam e atualizem o mesmo saldo simultaneamente.

O saldo é recarregado enquanto a conta está travada. Assim, a segunda operação enxerga o saldo atualizado pela primeira e recusa a operação se não houver
saldo para `valor + tarifa`.

Nas transferências externas, as transações `EM_ANDAMENTO` também funcionam como reservas lógicas: seus valores são somados ao calcular o saldo disponível.
Não existe uma coluna separada de `saldo_reservado`.

O `CHECK(balance_cents >= 0)` permanece como última proteção do banco, mas a consistência depende principalmente dos locks e da validação dentro da
transação SQL.

Enviar para outro banco, com chave já existente

Depois de localizar a conta remetente, a API procura a chave de idempotência antes de consultar o banco externo ou verificar o saldo novamente.

- mesma chave e mesmo payload:
  - `CONCLUIDA`: retorna a transação salva;
  - `EM_ANDAMENTO`: consulta o banco externo e tenta reconciliar;
  - `FALHOU`: retorna a rejeição registrada;
- mesma chave com payload incompatível: responde `422 QIT001022`.

A repetição não cria uma nova transação externa.

Enviar para outro banco, caminho feliz

1. A API valida a conta remetente local.
2. Consulta o banco externo para validar a conta destinatária.
3. Trava a conta remetente.
4. Refaz as validações de estado, idempotência e saldo, incluindo as reservas de operações `EM_ANDAMENTO`.
5. Cria a transação local como `EM_ANDAMENTO`.
6. Usa a `chave_transacao` local como identificador da operação no banco externo.
7. Confirma essa transação local e libera a conta.
8. Chama o banco externo sem manter o lock local.
9. Quando o banco externo responde `SETTLED`, a API trava a transação e a conta remetente, debita `valor + tarifa`, cria o lançamento de débito e marca a
   transação como `CONCLUIDA`.
10. Responde `201` na primeira conclusão ou `200` quando a operação é recuperada por retry.

A contraparte externa fica registrada em `recipient_bank_id`, `recipient_branch` e `recipient_number`.

Enviar para outro banco, rejeição

Se o banco externo responder `REJECTED`:

1. A transação local é marcada como `FALHOU`.
2. Nenhum débito ou crédito é criado no extrato local.
3. A API responde `422 QIT001019`.

Como o débito local só ocorre depois de `SETTLED`, não é necessário criar um lançamento de `CREDITO` compensatório.

Enviar para outro banco, timeout

Se o banco externo não responder dentro do timeout:

1. A transação local permanece `EM_ANDAMENTO`.

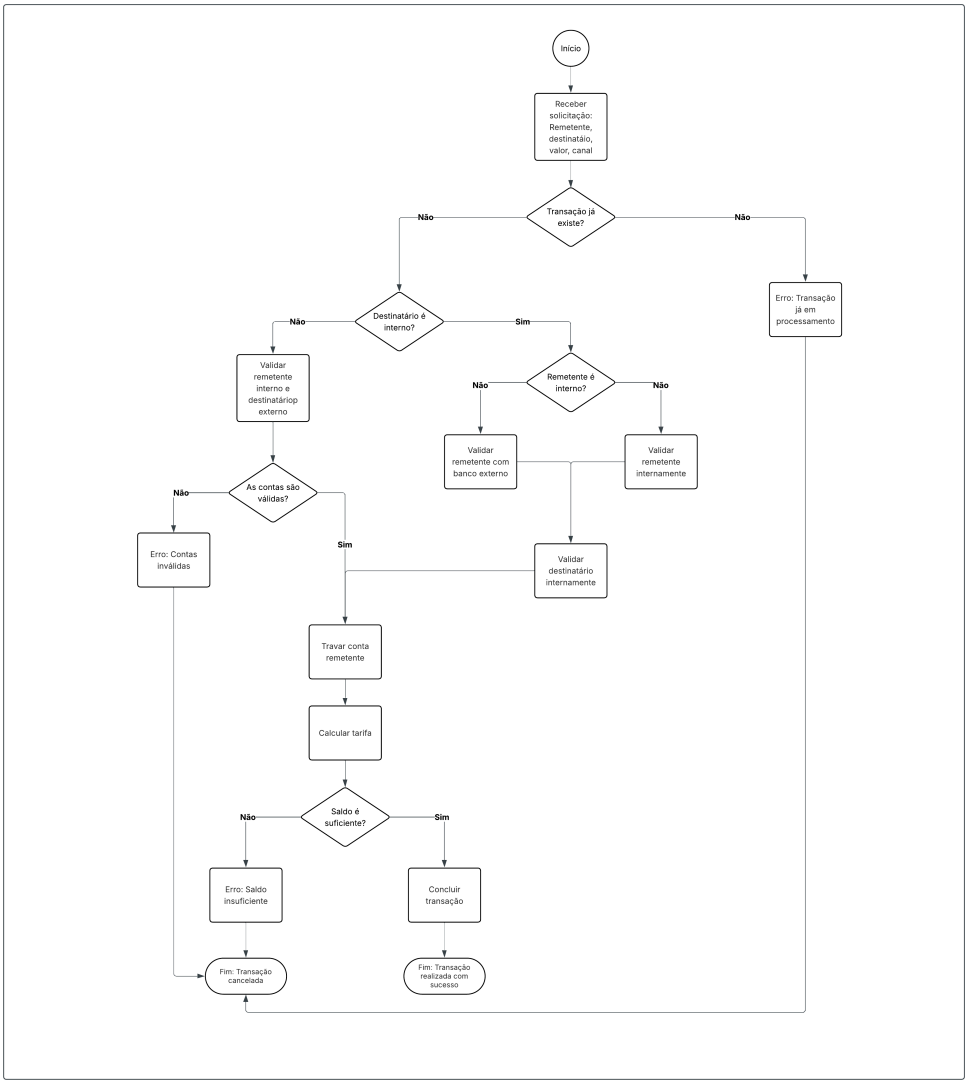
2. Nenhum lançamento local é criado naquele momento.
3. A API responde `504 QIT001020`.
4. Não existe retry automático em background.
5. O cliente deve guardar a mesma `chave_de_idempotencia` e repetir o `POST`.
6. A API consulta o banco externo usando o mesmo `external_transaction_id`.
7. Se a operação estiver liquidada, cria o débito local e marca a transação como `CONCLUIDA`.

O timeout não significa que a operação falhou.

Se o banco externo já tiver respondido `SETTLED`, mas a efetivação local falhar, a transação permanece `EM_ANDAMENTO`. Um novo retry usando a mesma chave
tenta novamente a liquidação local sem criar outra operação externa.

Nesta implementação, o timeout responde `504`, e não `202`. O estado pendente é persistido no banco e pode ser recuperado pela chave de idempotência.

Diagrama de fluxo da transação



Pagamento por boleto

Decisões de projeto

- O `BankSlip Mock` representa a infraestrutura externa de registro, consulta e liquidação de boletos.
- Nosso banco gerencia exclusivamente suas contas, reservas, transações e lançamentos.

- O beneficiário externo não precisa possuir conta local.
- A comunicação HTTP externa nunca ocorre enquanto travas de contas estiverem ativas.
- A reserva de saldo é efetiva e considerada por todas as demais operações financeiras.
- Boletos não possuem tarifa.
- O pagamento não reutiliza automaticamente as validações de destinatário de PIX/TED.

Pagar boleto, caminho feliz

1. Recebe a conta pagadora, a chave do boleto e a chave de idempotência. Consulta o boleto no `BankSlipConnector` e valida existência, vencimento, situação e dados do beneficiário. Retorna `404` `QIT001024` para boleto inexistente, `409` `QIT001025` para vencido e `409` `QIT001026` para boleto já pago.
2. Se o boleto possuir beneficiário interno, verifica a consistência entre o registro local e os dados externos. Divergências retornam `409` `QIT001030`.
3. Trava a conta pagadora e revalida saldo disponível, estado da conta e duplicidade da operação. Uma tentativa incompatível em andamento retorna `409` `QIT001027`.
4. Com saldo suficiente, reserva efetivamente o valor e cria uma `Transacao` do canal `BOLETO`, em `EM_ANDAMENTO`, persistindo a chave de idempotência antes da comunicação externa. A reserva impede que outras operações consumam o valor comprometido.
5. Faz commit, libera as travas e solicita o pagamento ao `BankSlipConnector`. Nenhuma trava local permanece ativa durante a chamada HTTP.
6. Após receber `SETTLED`, recupera a transação local e trava as contas envolvidas, em ordem determinística.
7. Na mesma transação SQL:
   - Persiste a confirmação e o identificador de liquidação externo;
   - Confirma a reserva e efetiva o débito do pagador;
   - Credita o beneficiário, **caso seja interno;**
   - Cria os lançamentos assinados correspondentes;
   - Atualiza os saldos e marca a transação como `CONCLUIDA`.
8. Faz commit e retorna `201` com os dados da transação concluída. O pagamento de boleto não possui tarifa.

Pagar boleto, falha: serviço externo rejeita a operação

1. A transação local já está registrada como `EM_ANDAMENTO`, com o valor reservado.
2. O `BankSlipConnector` retorna uma rejeição definitiva (`REJECTED`).
3. Nosso banco trava a conta pagadora, libera a reserva e marca a transação como `FALHOU`, atomicamente.
4. Não gera lançamentos financeiros e retorna `409` `QIT001029`.

Pagar boleto, falha: timeout ou resultado desconhecido

1. Nosso banco envia a solicitação de pagamento utilizando a chave de idempotência previamente persistida.
2. O serviço externo não responde dentro do prazo. Retorna `504` `QIT001028`, mas mantém a transação como `EM_ANDAMENTO` e o valor reservado.
3. A operação é encaminhada para conciliação, utilizando o mesmo identificador da tentativa original.
4. Se a liquidação for confirmada, executa a finalização financeira normalmente. Se houver rejeição definitiva, libera a reserva e marca a transação como `FALHOU`.
5. Enquanto o resultado externo for desconhecido, nenhuma nova movimentação financeira é realizada.

Pagar boleto, concorrência: duas solicitações simultâneas

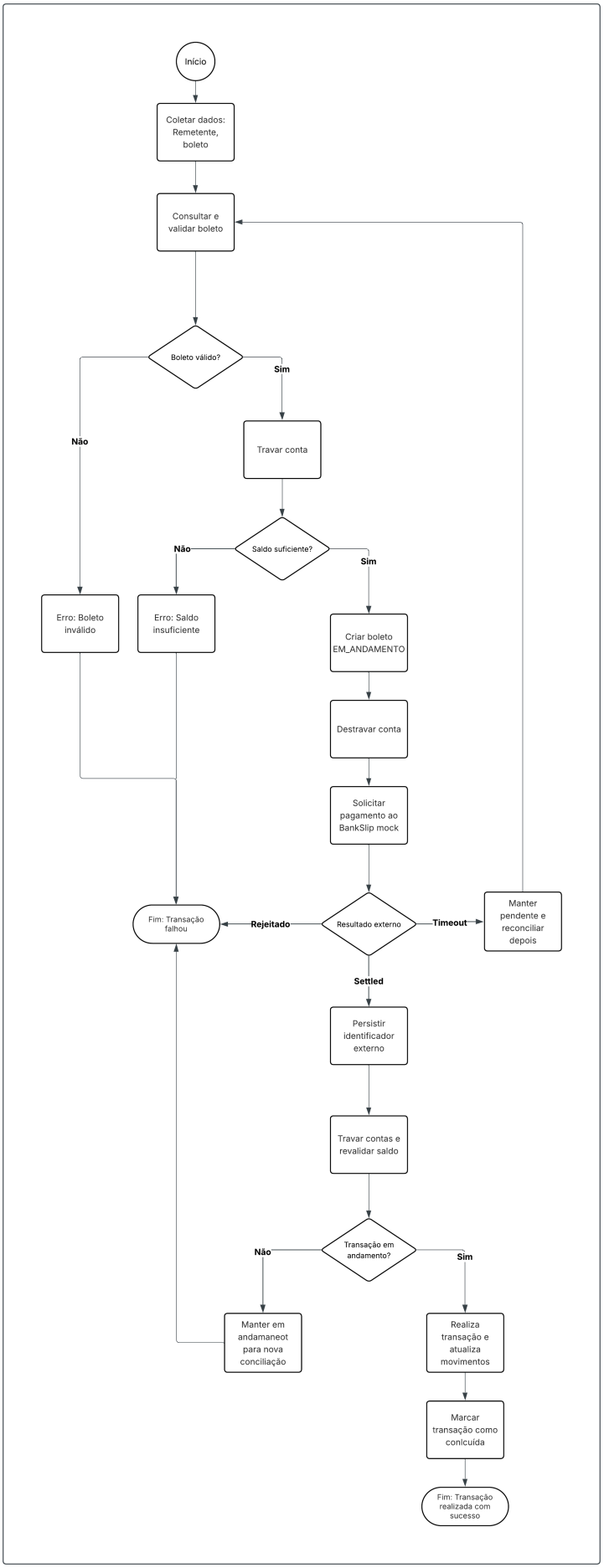
1. Os pedidos A e B tentam pagar o mesmo boleto.
2. A obtém a trava, verifica a disponibilidade e registra a transação `EM_ANDAMENTO`, com reserva de saldo.
3. B encontra a operação já registrada. Caso utilize outra chave de idempotência, recebe `409` `QIT001027`.

4. Se B repetir exatamente a mesma solicitação com a mesma chave, recupera a operação existente, sem criar outra transação ou reserva.
5. Após a liquidação de A, novas tentativas incompatíveis recebem `409` `QIT001026`.

Conciliação e idempotência da liquidação

1. Uma transação `EM_ANDAMENTO` pode ser recuperada após timeout, falha de persistência local ou interrupção do processo.
2. A conciliação consulta o resultado externo utilizando o identificador original.
    a. Neste projeto, a conciliação é disparada pelo retry utilizando a mesma chave de idempotência.
3. Antes de criar lançamentos, verifica se a transação já foi concluída.
4. A confirmação externa, os lançamentos, as alterações de saldo e a transição para `CONCLUIDA` são persistidos atomicamente.
5. A mesma confirmação externa nunca pode gerar lançamentos ou alterações de saldo duplicados.
6. Se ocorrer uma inconsistência local após `SETTLED`, a operação permanece pendente para conciliação. Uma liquidação externa confirmada não pode ser simplesmente descartada ou marcada como falha.

Diagrama de fluxo do pagamento por boleto


Início
Coletar dados: Remetente, boleto
Consultar e validar boleto
Boleto válido?
Sim
Não
Travar conta
Saldo suficiente?
Não
Sim
Erro: Boleto inválido
Erro: Saldo insuficiente
Criar boleto EM_ANDAMENTO
Destravar conta
Solicitar pagamento ao BankSlip mock
Fim: Transação falhou
Rejeitado
Resultado externo
Timeout
Manter pendente e reconciliar depois
Settled
Persistir identificador externo
Travar contas e revalidar saldo
Transação em andamento?
Não
Sim
Manter em andamaneot para nova conciliação
Realiza transação e atualiza movimentos
Marcar transação como conlcuída
Fim: Transação realizada com sucesso

Consulta do extrato

Extrato, caminho feliz

1. O cursor `proxima` contém a `chave_transacao` do último item retornado na página anterior.
2. A API procura essa transação junto de um `account_entry` pertencente à conta consultada.
3. Se o cursor for válido, a API lê `limite + 1` lançamentos sem travar a conta.
4. Os lançamentos são ordenados por `created_at DESC, id DESC`.
5. O item adicional indica se existe próxima página.
6. Se existir, `proxima` recebe a `chave_transacao` do último item retornado. Caso contrário, recebe `null`.

A paginação é baseada em cursor, não em número de página. Portanto, um lançamento novo no topo não desloca os itens que ainda serão buscados a partir do
cursor.

O extrato consulta `account_entry` e usa `bank_transaction` para recuperar o contexto da operação. O campo `movimentacao` é assinado:

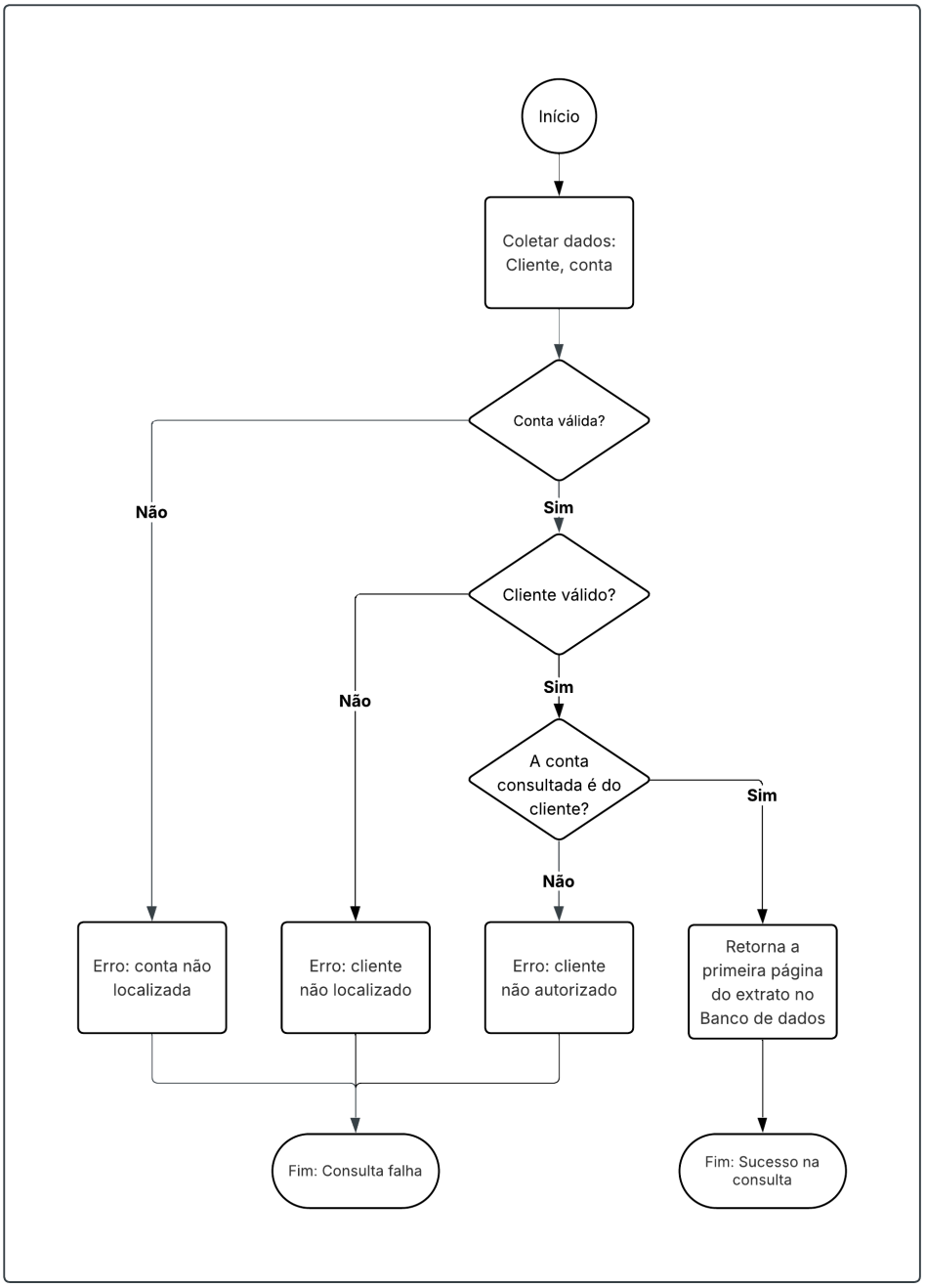
- débito do remetente: `(valor + tarifa_cobrada)`;
- crédito de transferência interna: `+valor`;
- envio externo rejeitado: nenhum lançamento;
- boleto pago: débito do pagador e, quando aplicável, crédito do beneficiário interno.

O lançamento não armazena `saldo_apos`. O saldo atual da conta é mantido em `account.balance_cents`; portanto, o extrato não deve afirmar diretamente qual
era o saldo após cada lançamento.

Extrato, falha: cursor inválido

1. A `proxima` não existe ou pertence a outra conta.
2. A API não encontra um `account_entry` correspondente à conta consultada.
3. A API responde `400 QIT001030`.
4. A página não é consultada.

Diagrama de fluxo da consulta de extrato



> **Principal desafio**
>
> **Qual é:** enviar dinheiro para outro banco sem perder nem duplicar um centavo quando o outro banco demora ou não responde.
>
> **Por que é difícil:** a saída mais simples é fazer tudo numa transação e responder `502` se a chamada falhar. Isso desfaz o débito aqui, mas o outro banco pode já ter creditado. O cliente lê o `502` como nada feito, faz uma ordem nova, e o dinheiro sai duas vezes. Travar a conta durante a chamada pararia a movimentação dela por até 5 s, e chamar o outro banco antes de debitar deixaria a conta gastar, no intervalo, o dinheiro já enviado.
>
> **Como o desenho resolve:** o envio usa duas transações no banco de dados, sem trava durante a chamada. A primeira trava a conta, confere o saldo e confirma o `DEBITO`, `TARIFA` e a transação `EM_ANDAMENTO`, que o extrato já mostra. A chamada leva a `chave_transacao` como `e2e_id`, e com ele o outro banco não executa a mesma ordem duas vezes. Com a resposta, a segunda grava `CONCLUIDA`, ou `FALHOU` com um lançamento que devolve valor e tarifa, do tipo `CREDITO`. Sem resposta, o cliente recebe `202`, repete o `POST` com a mesma chave, e a API retoma da chamada. Se ninguém repetir, o envio fica `EM_ANDAMENTO` e o débito reservado continua no extrato. Uma conciliação, que está fora do escopo, só precisaria reenviar a ordem com o mesmo `e2e_id`, e o outro banco não moveria o dinheiro de novo.